

LIBRARY MANAGEMENT SYSTEM USING C & DATA STRUCTURES

LEVEL 1 — MENU & PROGRAM STRUCTURE

1. Objective

The objective of Level 1 is to create the basic framework of the Library Management System.

Students will learn:

- main() function
- User-defined functions
- printf() and scanf()
- switch-case
- do-while loop
- Menu-driven programming
- Basic program modularity

Important: No data structure is introduced in Level 1. We are only creating the skeleton of the project.

2. Data Structure Used

None

C Concepts Used

- main()
- Starting point
- Function
- Separate menu module
- switch
- Select operation
- do-while
- Repeat menu
- Variable
- Store user's choice
- printf()
- Display information
- scanf()
- Take input

3. Problem Statement

Create a menu-driven C program for a Library Management System.

The system should provide the following options:

1. Add Book
2. Display Books
3. Search Book
4. Issue Book
5. Return Book
6. Exit

At Level 1, these options only display a message. Actual book management will be implemented in later levels.

4. Algorithm

Step 1: Start the program.

Step 2: Display the Library Management System menu.

Step 3: Read the user's choice.

Step 4: Use switch-case to identify the selected operation.

Step 5: Display the corresponding message.

Step 6: Repeat the menu until the user selects Exit.

Step 7: Stop the program.

5. C Program

```
#include <stdio.h>
```

```
void displayMenu()
```

```
{
    printf("\n=====\\n");
    printf("    LIBRARY MANAGEMENT SYSTEM\\n");
    printf("=====\\n");

    printf("1. Add Book\\n");
    printf("2. Display Books\\n");
    printf("3. Search Book\\n");
    printf("4. Issue Book\\n");
    printf("5. Return Book\\n");
    printf("6. Exit\\n");

    printf("=====\\n");
}
```

```
int main()
```

```
{
    int choice;

    do
    {
        displayMenu();

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                printf("Add Book selected.\\n");
                break;

            case 2:
                printf("Display Books selected.\\n");
                break;

            case 3:
                printf("Search Book selected.\\n");
                break;

            case 4:
                printf("Issue Book selected.\\n");
```

```

        break;

    case 5:
        printf("Return Book selected.\n");
        break;

    case 6:
        printf("Thank you for using the Library System.\n");
        break;

    default:
        printf("Invalid choice! Please try again.\n");
    }

} while(choice != 6);

return 0;
}

```

6. Sample Input

Enter your choice: 1

Sample Output

```
=====
```

LIBRARY MANAGEMENT SYSTEM

```
=====
```

1. Add Book
2. Display Books
3. Search Book
4. Issue Book
5. Return Book
6. Exit

```
=====
```

Enter your choice: 1

Add Book selected.

Another example:

Enter your choice: 7

Output:

Invalid choice! Please try again.

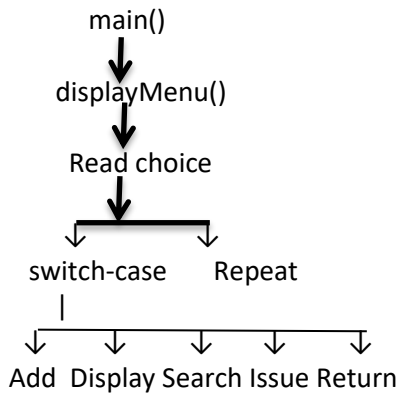
Exit:

Enter your choice: 6

Thank you for using the Library System.

7. What Students Should Understand

At the end of Level 1, students should understand this structure:



"We are not building the complete Library Management System in one program. We are building it step by step. Level 1 creates the skeleton, and every next level will add one new data structure or algorithm."

8. Viva Questions

- What is a menu-driven program?
- What is the purpose of the `main()` function?
- Why did we use a do-while loop?
- What is the purpose of switch-case?
- What is the difference between while and do-while?
- What is a user-defined function?
- Why did we create `displayMenu()` separately?
- What does `break` do inside a switch?
- What happens when the user enters an invalid choice?
- Why does the program continue until choice 6?
- Data Structures Question
- Is any data structure used in Level 1?

LEVEL 2 — BOOK STRUCTURE

Level 1 = Program framework

Level 2 = Represent one book

1. Objective

The objective of Level 2 is to represent a library book using a structure in C.

Students will learn:

- struct
- typedef
- Structure members
- Character arrays
- Structure variables
- Accessing structure members
- Passing a structure to a function

2. Data Structure Used

Structure

We create:

```
typedef struct
{
    int id;
    char title[50];
    char author[50];
    int year;
    int available;
} Book;
```

A single Book contains:

```
Book
├── ID
├── Title
├── Author
├── Year
└── Availability
```

3. Problem Statement

Modify the Level 1 Library Management System so that the user can enter information about one book.

The system should store:

- Book ID
- Book Title
- Author
- Publication Year
- Availability

and then display the complete book information.

4. Algorithm

Add Book

START

↓

Create Book structure

↓

Read Book ID

↓

Read Title

↓

Read Author

↓

Read Year
↓
Set availability = Available
↓
Display Book
↓
STOP

5. C Program

```
#include <stdio.h>
typedef struct
{
    int id;
    char title[50];
    char author[50];
    int year;
    int available;
} Book;

void addBook(Book *b)
{
    printf("\nEnter Book ID: ");
    scanf("%d", &b->id);
    printf("Enter Book Title: ");
    scanf(" %49[^\n]", b->title);
    printf("Enter Author Name: ");
    scanf(" %49[^\n]", b->author);
    printf("Enter Publication Year: ");
    scanf("%d", &b->year);

    b->available = 1;
    printf("\nBook added successfully!\n");
}

void displayBook(Book b)
{
    printf("\n===== \n");
    printf("      BOOK DETAILS\n");
    printf("===== \n");

    printf("Book ID    : %d\n", b.id);
    printf("Title     : %s\n", b.title);
    printf("Author    : %s\n", b.author);
    printf("Year      : %d\n", b.year);

    if(b.available == 1)
        printf("Status    : Available\n");
    else
        printf("Status    : Issued\n");

    printf("===== \n");
}

int main()
{
    Book book;
```

```

addBook(&book);
displayBook(book);
return 0;
}

```

6. Sample Input

Enter Book ID: 101
 Enter Book Title: Data Structures
 Enter Author Name: Lipschutz
 Enter Publication Year: 2024

7. Sample Output

Book added successfully!

```

=====
                BOOK DETAILS
=====
Book ID      : 101
Title       : Data Structures
Author      : Lipschutz
Year        : 2024
Status      : Available
=====

```

8. Understanding the Structure

Book book; Create one variable named book of type Book.

book.id; Access the id member of the structure.

But inside:

void addBook(Book *b)

we use:

b->id

because b is a pointer to a structure.

9. Viva Questions Structure

- What is a structure in C?
- Why do we use a structure for a book?
- How is a structure different from an array?
- What is typedef?
- What is a structure member?
- How do you access a structure member?
- What is the difference between . and ->?
- Why are title and author declared as character arrays?
- What does available = 1 represent?
- What happens if available = 0?
- Pointer-related
- Why is Book *b used in addBook()?
- What does &book mean?
- Why do we use b->id instead of b.id?
- What happens if we pass book instead of &book?
- Data Structures
- Why is a structure useful in a Library Management System?
- Can we store 100 books using only this structure?
- What data structure will we need to store multiple books?
- What will we introduce in Level 3?

LEVEL 3 — ARRAY OF BOOKS

This level extends Level 2, where we created a structure for one book. Now the problem is: How do we store 50 or 100 books?

Answer: Array of Structures.

1. Objective

By completing Level 3, students will learn to:

- Store multiple book records.
- Use an array of structures.
- Add multiple books.
- Display all books.
- Traverse an array.
- Understand indexing.
- Understand the relationship between structure and array.
- Analyze the basic time complexity of insertion and traversal.

Previous → Current

Level 2

One Book



struct Book



Level 3

Multiple Books



struct Book books[100]

2. Data Structure Used

Array of Structures

Book books[100];

Here:

Book → structure data type

books → array name

100 → maximum number of books

Conceptually:

books[0] → Book 1

books[1] → Book 2

books[2] → Book 3

books[3] → Book 4

.

.

books[99] → Book 100

Each element contains:

Book



3. Problem Statement

Modify the Level 2 program so that the Library Management System can store multiple books instead of only one book.

The program should:

Ask how many books the user wants to add.

Accept details of each book.

Store them in an array.

Display all stored books.

4. Algorithm

Add Books

START

↓

Create Book array

↓

Read number of books

↓

i = 0

↓

Read book details

↓

Store in books[i]

↓

i = i + 1

↓

Are all books entered?

↓

NO —————> Read next book

↓ YES

Display all books

↓

STOP

5. C Program

```
#include <stdio.h>
```

```
#define MAX_BOOKS 100
```

```
typedef struct
```

```
{  
    int id;  
    char title[50];  
    char author[50];  
    int year;  
    int available;
```

```
} Book;
```

```
/* Function to add books */
```

```
void addBooks(Book books[], int n)
```

```
{  
    int i;  
  
    for(i = 0; i < n; i++)  
    {  
        printf("\nEnter details of Book %d\n", i + 1);  
        printf("Book ID: ");  
        scanf("%d", &books[i].id);  
        printf("Book Title: ");  
        scanf(" %49[^\n]", books[i].title);  
        printf("Author Name: ");  
        scanf(" %49[^\n]", books[i].author);  
        printf("Publication Year: ");  
        scanf("%d", &books[i].year);
```

```
/* Initially every book is available */  
books[i].available = 1;
```

```

    }
}

/* Function to display all books */
void displayBooks(Book books[], int n)
{
    int i;

    printf("\n=====\\n");
    printf("        LIBRARY BOOK LIST\\n");
    printf("=====\\n");

    for(i = 0; i < n; i++)
    {
        printf("\\nBook %d\\n", i + 1);

        printf("Book ID    : %d\\n", books[i].id);
        printf("Title     : %s\\n", books[i].title);
        printf("Author    : %s\\n", books[i].author);
        printf("Year      : %d\\n", books[i].year);

        if(books[i].available == 1)
            printf("Status    : Available\\n");
        else
            printf("Status    : Issued\\n");

        printf("-----\\n");
    }
}

```

```

int main()
{
    Book books[MAX_BOOKS];

    int n;

    printf("Enter number of books: ");
    scanf("%d", &n);

    if(n <= 0 || n > MAX_BOOKS)
    {
        printf("Invalid number of books!\\n");
        return 0;
    }

    addBooks(books, n);

    displayBooks(books, n);

    return 0;
}

```

6. Sample Input

Enter number of books: 3

Enter details of Book 1

Book ID: 101
Book Title: Data Structures
Author Name: Lipschutz
Publication Year: 2024

Enter details of Book 2
Book ID: 102
Book Title: C Programming
Author Name: Dennis Ritchie
Publication Year: 2015

Enter details of Book 3
Book ID: 103
Book Title: Operating System
Author Name: Galvin
Publication Year: 2023

7. Sample Output

```
=====
                LIBRARY BOOK LIST
=====
```

```
Book 1
Book ID   : 101
Title     : Data Structures
Author    : Lipschutz
Year      : 2024
Status    : Available
-----
```

```
Book 2
Book ID   : 102
Title     : C Programming
Author    : Dennis Ritchie
Year      : 2015
Status    : Available
-----
```

```
Book 3
Book ID   : 103
Title     : Operating System
Author    : Galvin
Year      : 2023
Status    : Available
-----
```

8. Time Complexity

Adding books

If n books are entered:
Time Complexity = $O(n)$ because every book is processed once.

Displaying books

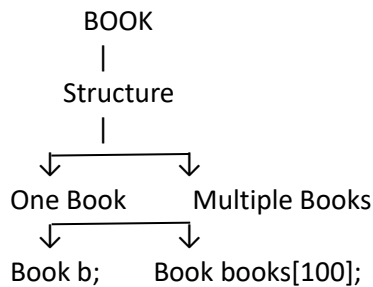
Time Complexity = $O(n)$ because every book is visited.

Accessing a particular index

For example:
books[5]
Time Complexity = $O(1)$ because an array provides direct indexing.

9. What Data Structure Have We Learned?

At this point students have learned:



So the important distinction is:

10. Exercises for Students

Exercise 1: Modify the program to display only the available books.

Exercise 2: Add a category field:

`char category[30];`

Exercise 3: Add a price field:

`float price;`

Exercise 4: Display the total number of books.

Exercise 5: Display the number of available books.

Exercise 6: Display the oldest book based on publication year.

Exercise 7: Change the maximum capacity from 100 to 500.

11. Related Viva Questions

- What is an array?
- What is an array of structures?
- Why do we need an array in this project?
- What is the maximum number of books in this program?
- What is the index of the first book?
- What is the index of the last book if there are 10 books?
- Structure + Array
- What is the difference between `Book books[100]` and `Book book`?
- How do you access the ID of the third book?
- What does `books[i].title` mean?
- Can an array store different data types?
- Why is an array of structures suitable for books?
- Complexity
- What is the complexity of displaying all books?
- What is the complexity of accessing `books[50]`?
- Why is array indexing $O(1)$?
- What is the space complexity of `Book books[100]`?
- Why don't we use a linked list at this stage?

Expected answer:

We are first learning how multiple records can be stored using an array of structures. A linked list will be introduced later when we need dynamic memory allocation and operations such as insertion and deletion.